Target Link:

https://www.saucedemo.com

Exercise 1: Resource Files and Variable Encapsulation
Goal: Create a clean separation between locators and test logic.
The Task: Create a `login_resources.resource` file.
Implementation: Define `${USERNAME_FIELD}`, `${PASSWORD_FIELD}`, and `${LOGIN_BUTTON}` in the ` Variables ` section. Create a keyword `Login To Application` that takes `${username}` and `${password}` as arguments.
The Test: Write a test case in a `.robot` file that imports the resource and logs in.
Verification: Verify that the "Products" title is visible after login.

Exercise 2: Custom Python Libraries for POM
Goal: Extend Robot Framework with a Python-based Page Object.
The Task: Write a Python class `ProductPage.py`. Inside, create a method `get_product_price_by_name(product_name)` that returns the price of a specific item.
Implementation: Import this Python file as a `Library` in your Robot script.
The Test: Log in, then use your custom keyword to fetch the price of "Sauce Labs Backpack" and "Sauce Labs Bike Light".
Verification: Use `Should Be Equal As Numbers` to verify the total sum of these two items is correct.

Exercise 3: Setup/Teardown with Screenshots
Goal: Manage browser lifecycle and failure evidence.
The Task: Implement a `Suite Setup` to open the browser and a `Test Teardown` to capture evidence.
Implementation: In the teardown, use the `Run Keyword If Test Failed` keyword to trigger `Capture Page Screenshot`.
Challenge: Configure the screenshot name to include the test name and a timestamp (e.g., `failure_${TEST_NAME}.png`).

Exercise 4: Data-Driven Testing with Scenario Outlines
Goal: Test multiple login credentials using a Template.
The Task: Use the `Test Template` feature in Robot Framework.
Implementation: Create a keyword `Invalid Login Scenario`.
The Data: Provide a table of `invalid_user`, `locked_out_user`, and `problem_user` with their respective error messages.
Verification: Ensure that for each row, the correct error message (e.g., "Epic sadface: Username and password do not match any user in this service") appears.

Exercise 5: Advanced Reporting with Robot & Allure
Goal: Integrate Allure for high-level visualization.
The Task: Install the `robotframework-allure` listener.

Implementation: Add tags to your test cases like `[Tags] Critical Smoke Regression`.

Execution: Run your tests using the command line flag to generate Allure results: `robot --listener allure_robotframework .`

Challenge: Use the `Set Test Documentation` keyword within your test to provide a detailed description that will show up in the Allure report dashboard.

```
Robot_Project/
│
├── Resources/
│   ├── common.resource       # Browser setup/teardown
│   ├── login_page.resource   # Login locators and keywords
│   └── cart_page.resource    # Cart locators
│
├── Libraries/
│   └── InternalCalculations.py # Custom Python logic
│
├── Tests/
│   ├── smoke_tests.robot
│   └── regression_tests.robot
│
├── Results/              # Standard Robot logs
└── Allure-Results/       # For Allure reporting
```